

영화 정보 알리미 (KinoLetter)

202126861 김영빈

프로젝트 개요

- 영화를 좋아하는 사람으로서 극장가에 사람들의 발길이 끊기는 것이 아쉽다. 가격의 문제도 크지만, 화제작이나 블록버스터 영화들은 여전히 비싼 가격을 감수하고 보러 가는 사람들이 많은 것을 보면 보고 싶은 영화에 대한 정보가 충분하지 않다는 점도 문제가 될 수 있을 것 같다고 생각했다.
- 따라서 이번 프로젝트를 통해 사람들 개개인의 취향을 바탕으로 현재 상영 중인 영화나 개봉 예정인 영화의 정보를 추천해주는 서비스의 필요성을 느꼈다. 사용자가 '좋아요'를 누른 내용을 바탕으로 곧 개봉하는 영화들 중 사용자의 취향에 맞을 것 같은 영화를 소개해주고, 그 중 사용자가 '찜'한 영화의 개봉일에 맞춰 메일을 전송하는 시스템을 구현하고자 한다.

프로젝트 개요

- 영화진흥위원회 OpenAPI를 통해 영화 정보 및 박스오피스 순위 데이터를 수집하였다. 개봉 예정 영화는 별도 API가 없기 때문에, 등록된 영화 중 개봉일이 오늘 이후인 영화를 필터링하여 분류하였다.
- 백엔드는 Python과 Flask를 사용하여 간결하고 빠르게 REST API를 구축하였다. 프론트엔드는 HTML, CSS, JavaScript만을 사용하여 최소한의 UI로 핵심 기능에 집중하였으며, 데이터베이스 중시의 흐름 구현에 중점을 두었다.
- Oracle Cloud Infrastructure에 Ubuntu 인스턴스를 생성하고, 공인IP를 통해 외부 접속이 가능하도록 설정하였다. Flask 앱은 Gunicorn을 통해 실행되고, Nginx를 리버스 프록시로 구성하여 간단한 서비스 배포를 완료하였다. 도메인 연결, HTTPS 설정, 방화벽 세부 설정 등은 제외하고 기초적인 배포 환경 구성에 의의를 두었다.
- KinoLetter사이트 공인 IP : 129.154.48.98

제공 기능

1. 회원 가입 기능

- 이메일 주소, 비밀번호, 메일 수신 여부 정보를 기입

2. 영화 정보 검색 기능

- 제목을 검색해서 정보 확인 가능

3. 현재 상영작, 개봉 예정작, 박스오피스 순위 등 확인 기능

- 현재 상영작 탭에서 박스오피스 순위 정보 확인 가능
- 개봉 예정작 탭에서 개봉 예정 영화 확인 가능

4. '좋아요' 표시 기능

- 내가 좋아하는 영화에 '좋아요'를 표시하고, 선택 이유를 정할 수 있다.

제공 기능

5. 영화 추천 기능

- '좋아요' 영화의 '선택 이유'(감독, 장르, 국가, 배우 등)를 기반으로, 개봉 예정작 중 취향과 일치하는 영화를 추천 영화 목록 탭에서 확인할 수 있다.

6. '찜' 표시 기능

- 개봉 예정작을 찜할 수 있다.

7. '찜'한 영화 개봉 일주일 전 메일 전송 기능

- 회원 가입 시 입력한 이메일로 주 1회 해당 주의 개봉작 정보를 담은 메일을 전송해준다.
- 자동화를 구현하는데 한계가 있어서, 관리자가 직접 주 1회 (ex. 매주 일요일) 관리자 계정으로 접속하여 '메일 전송' 버튼을 누르는 것으로 구현하였다. Db에서 메일 전송 대상 계정을 찾아 일괄적으로 메일을 전송한다.

8. 회원 정보 수정 및 탈퇴

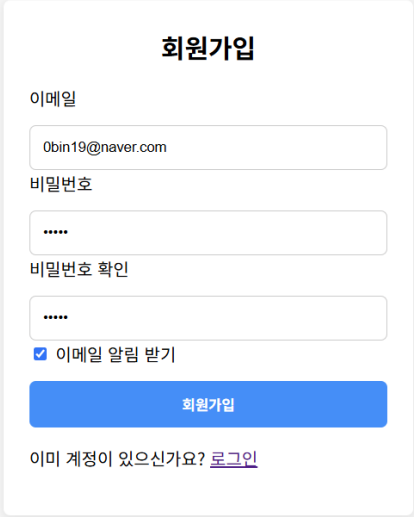
사용자 인터랙션 시나리오

- 좋아요 기능과 영화 추천

- 로그인한 사용자는 검색창을 이용해 기존에 좋아했던 영화에 '좋아요'를 표시할 수 있다.
- '현재 상영' 탭에서 인기작을 확인하고, '좋아요'를 표시할 수 있다.
- 사용자가 '좋아요'를 표시할 때마다 '좋아요'한 이유를 선택해야 한다. 선택된 이유들(감독, 장르, 배우 등)을 바탕으로, 개봉 예정작 중 사용자의 취향에 해당하는 작품을 추천한다.
- 사용자는 '추천 영화' 탭에서 마음에 드는 영화를 '찜'할 수 있다.

시나리오 #1. 사용자 (1)

- naver 계정으로 회원가입



회원가입

이메일
0bin19@naver.com

비밀번호
.....

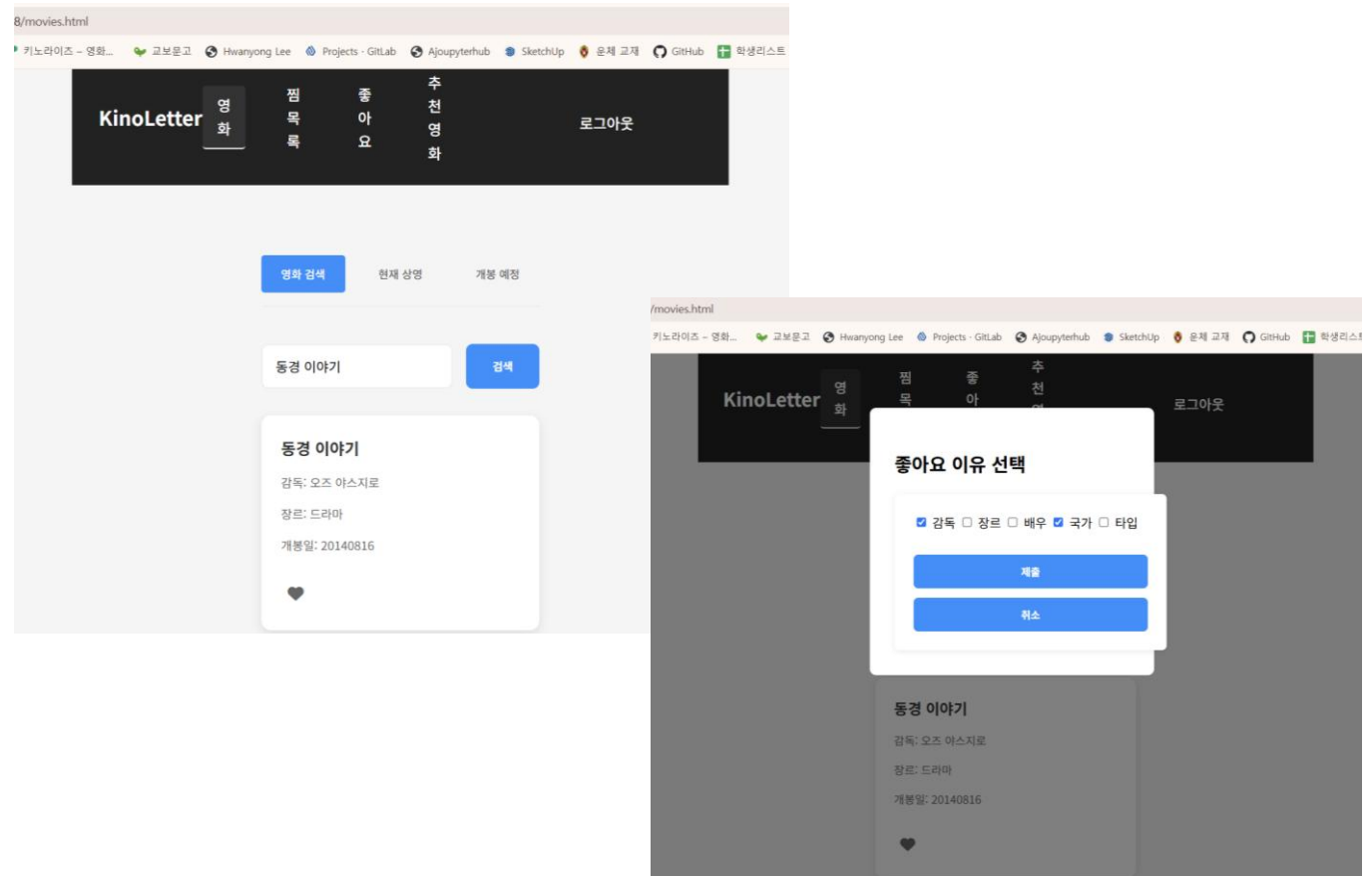
비밀번호 확인
.....

☒ 이메일 알림 받기

회원가입

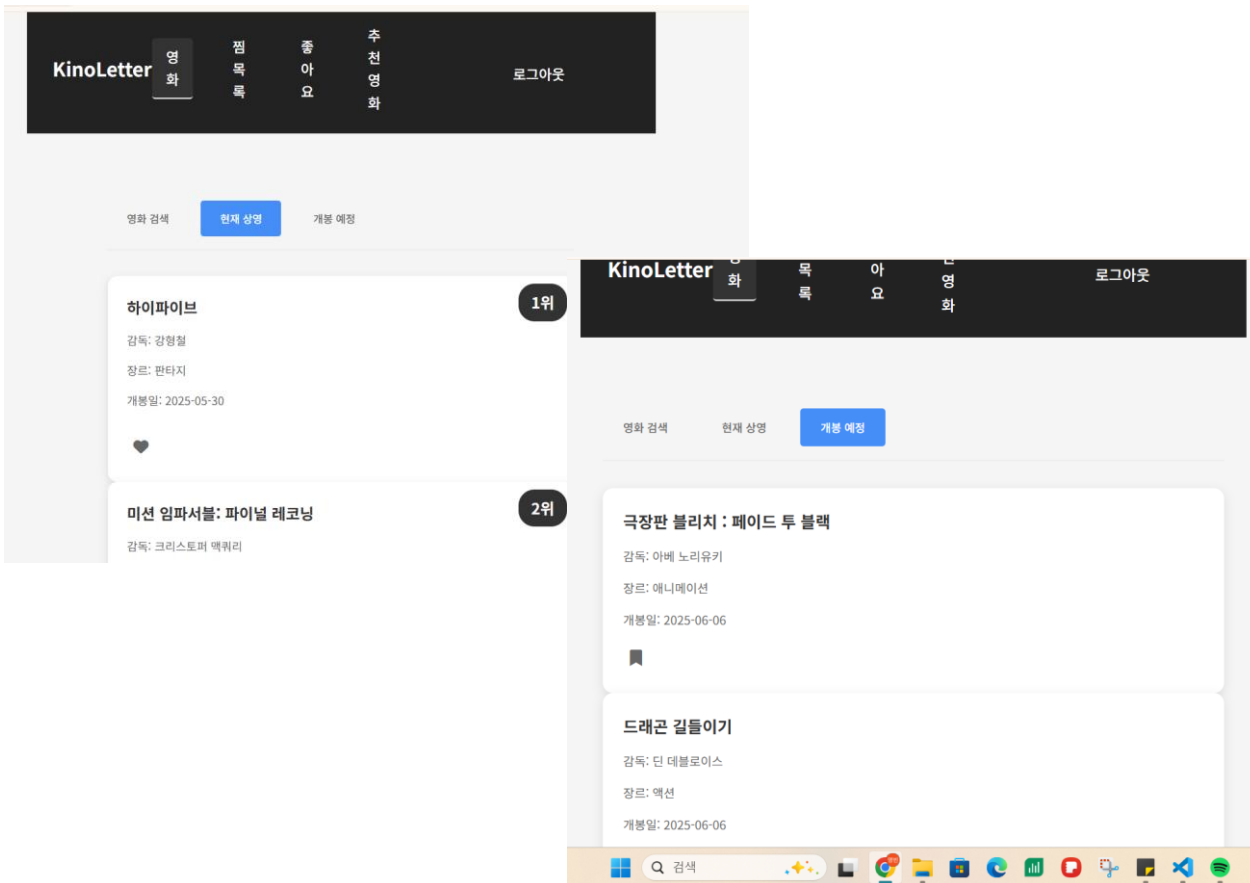
이미 계정이 있으신가요? [로그인](#)

- 영화정보 검색 및 좋아요

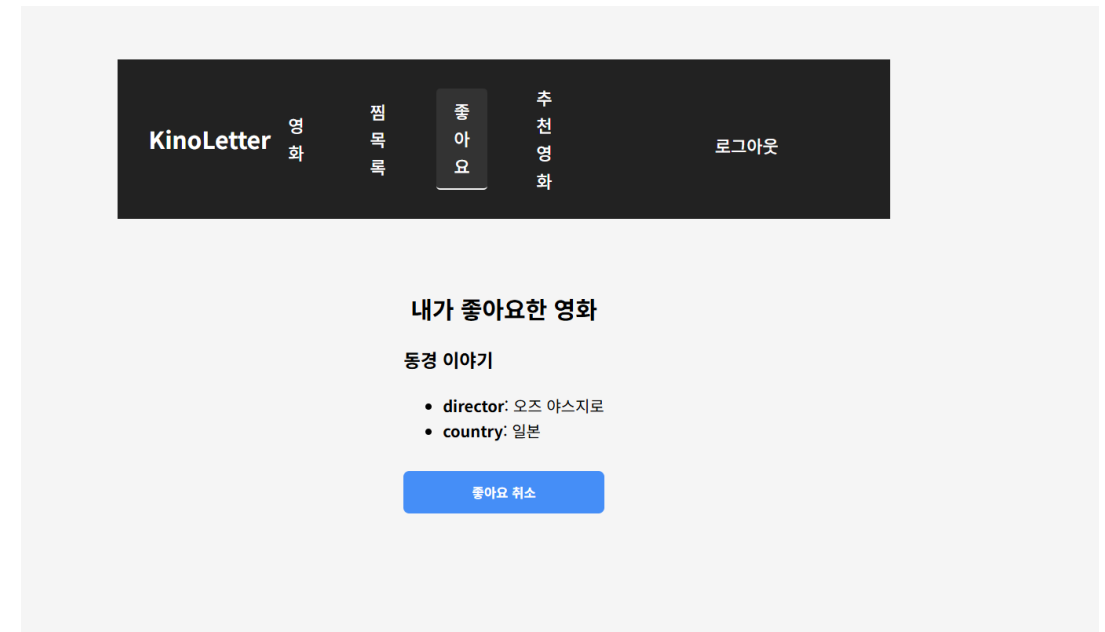


시나리오 #1. 사용자 (2)

- 현재상영 및 개봉예정 영화



- '좋아요' 탭에서 내가 좋아한 영화와 이유 확인



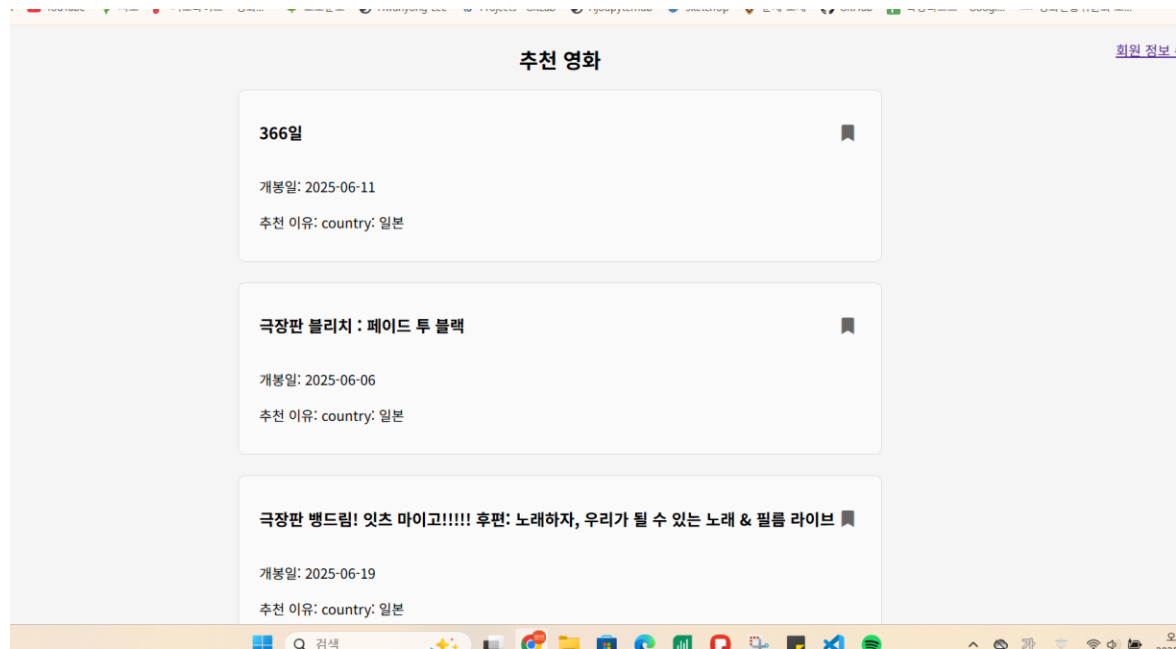
시나리오 #1. 사용자 (3)

- 좋아요 기반 영화 추천

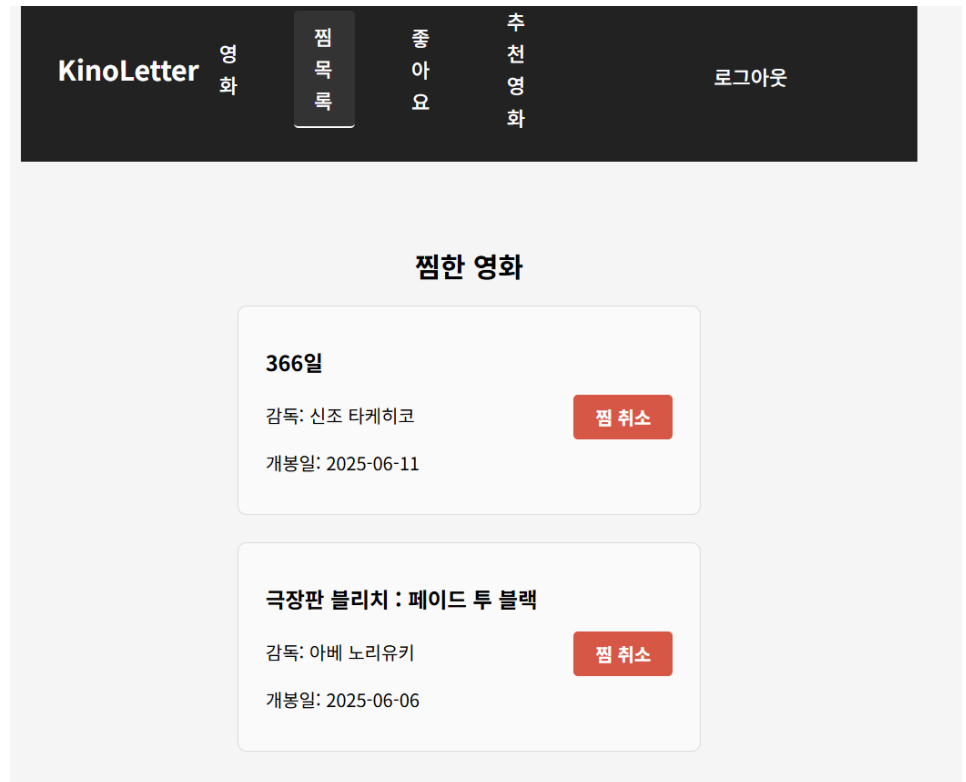
등록된 이유 =>

감독:오즈 야스지로

국가:일본

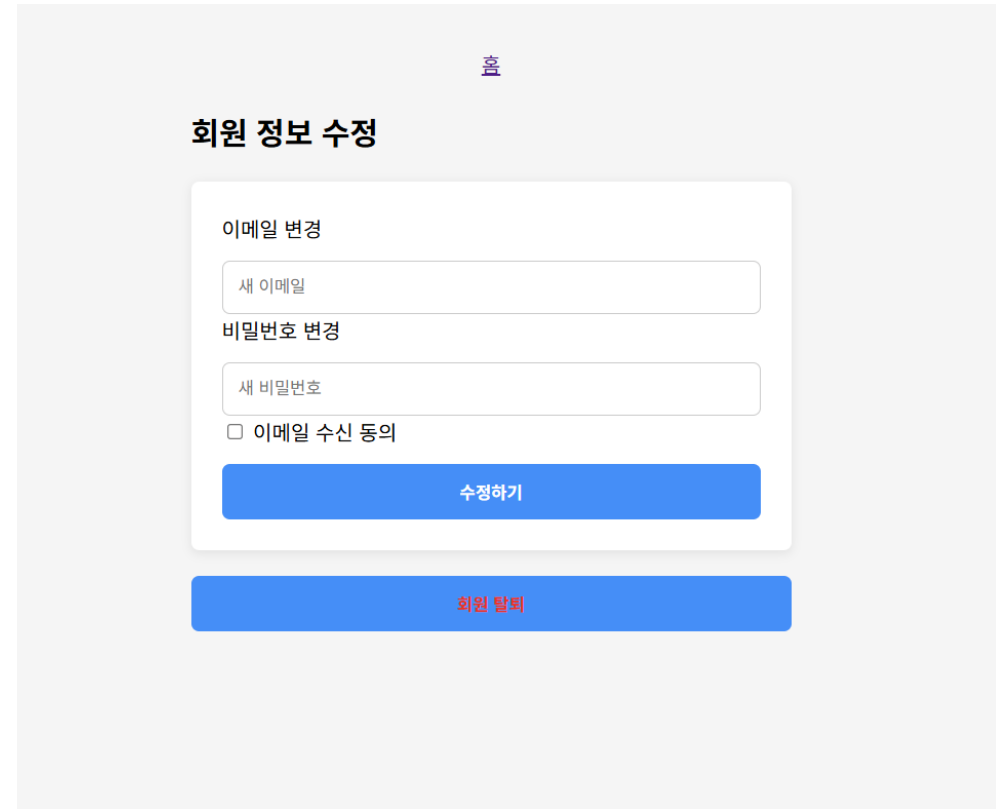
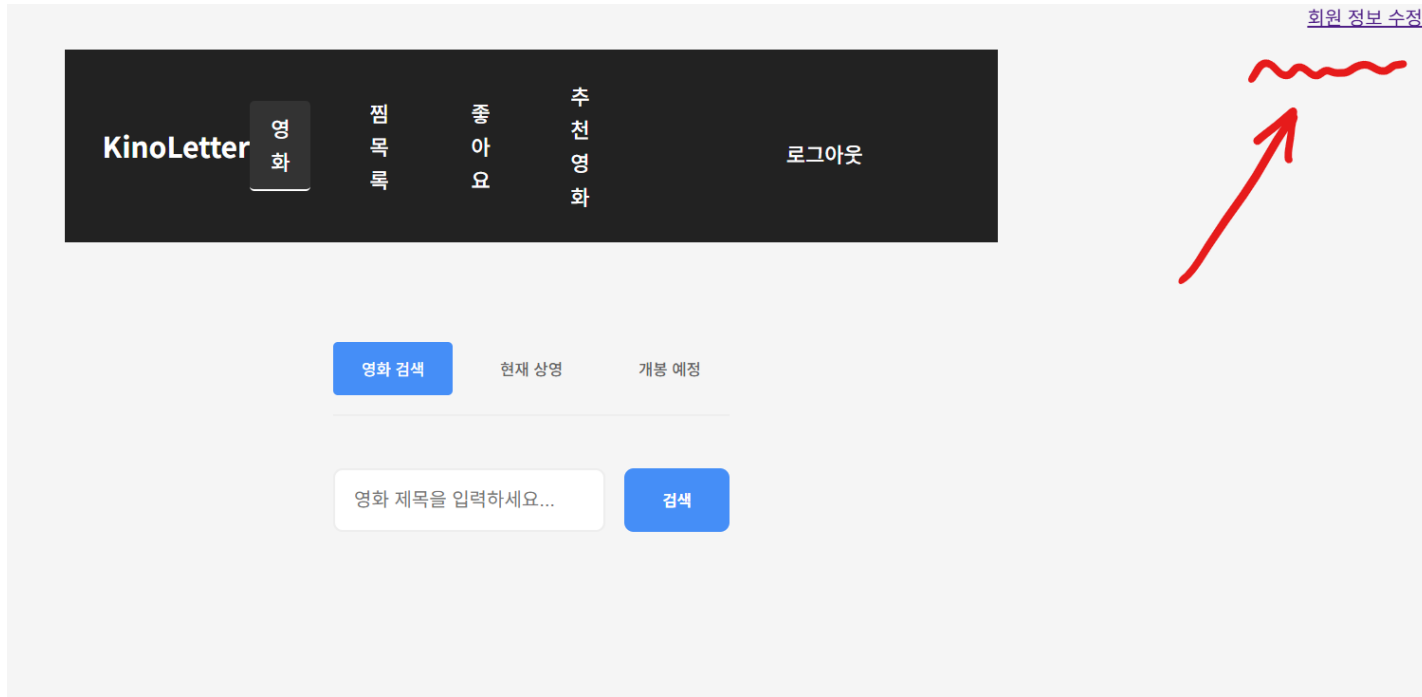


- 찜한 영화 목록 -> 메일 전송 대상



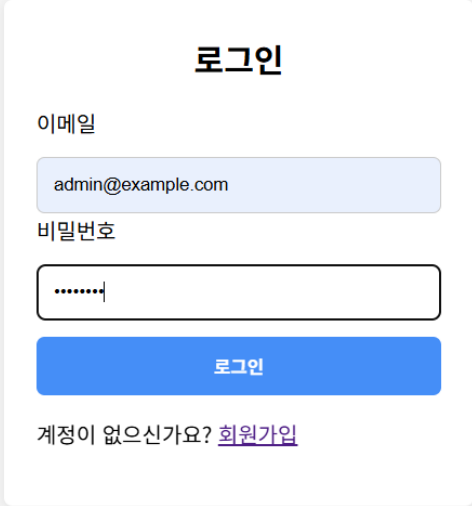
시나리오 #1. 사용자 (4)

- 회원 정보 수정 가능



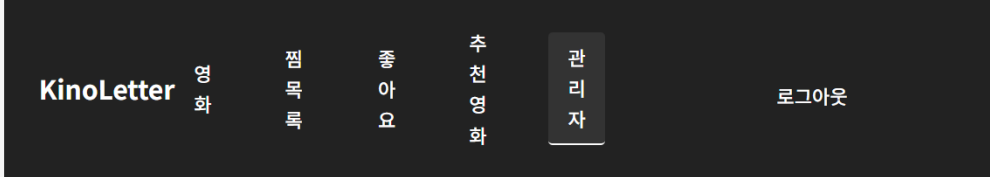
시나리오 #2. 관리자 (1)

- 관리자 계정으로 로그인



The login form is titled '로그인' (Login). It contains two input fields: '이메일' (Email) with the value 'admin@example.com' and '비밀번호' (Password) with masked characters '.....'. Below the password field is a blue '로그인' (Login) button. At the bottom, there is a link '계정이 없으신가요? [회원가입](#)' (Don't have an account? [Sign up](#)).

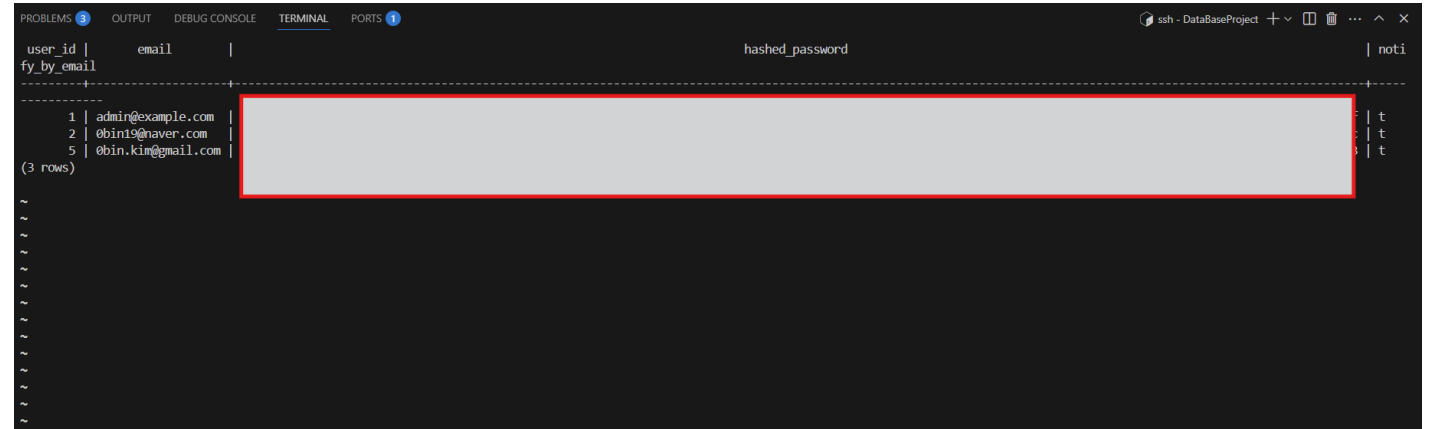
- 관리자 계정만 볼 수 있는 '관리자' 탭



The admin interface features a dark header bar with the 'KinoLetter' logo and navigation links: '영화' (Movies), '찜 목록' (Watchlist), '좋아요' (Like), '추천영화' (Recommended Movies), '관리자' (Admin), and '로그아웃' (Logout). The '관리자' link is highlighted with a white underline. Below the header, the section is titled '관리자 알림 발송' (Admin Notification Delivery). It includes a text prompt '버튼을 누르면 전체 회원에게 주간 추천 메일이 발송됩니다.' (Pressing the button will send a weekly recommendation email to all members.) and a dark button labeled '회원 전체 메일 발송' (Send email to all members).

시나리오 #2. 관리자 (2)

- select * from users;
현재 등록된 계정들 ->



user_id	email	hashed_password
1	admin@example.com	
2	0bin19@naver.com	
5	0bin.kin@gmail.com	

(3 rows)

- naver 계정으로 메일 도착

답장 전체답장 전달 삭제 스팸차단 안읽음 이동 더보기

☆ 다음 주 개봉 예정 영화 알림

보낸사람 kinoletter0@gmail.com

2025년 6월 6일 (금) 오후 3:55

다음 7일 내에 개봉 예정인 영화 목록입니다.

제목: 366일
개봉일: 2025-06-11
감독: 신조 타케히코
장르: 멜로/로맨스

제목: 극장판 블리치 : 페이드 투 블랙
개봉일: 2025-06-06
감독: 아베 노리유키
장르: 애니메이션

- google 계정으로 메일 도착

다음 주 개봉 예정 영화 알림 받은편지함

kinoletter0@gmail.com

나에게

오후 3:55 (14분 전)

다음 7일 내에 개봉 예정인 영화 목록입니다.

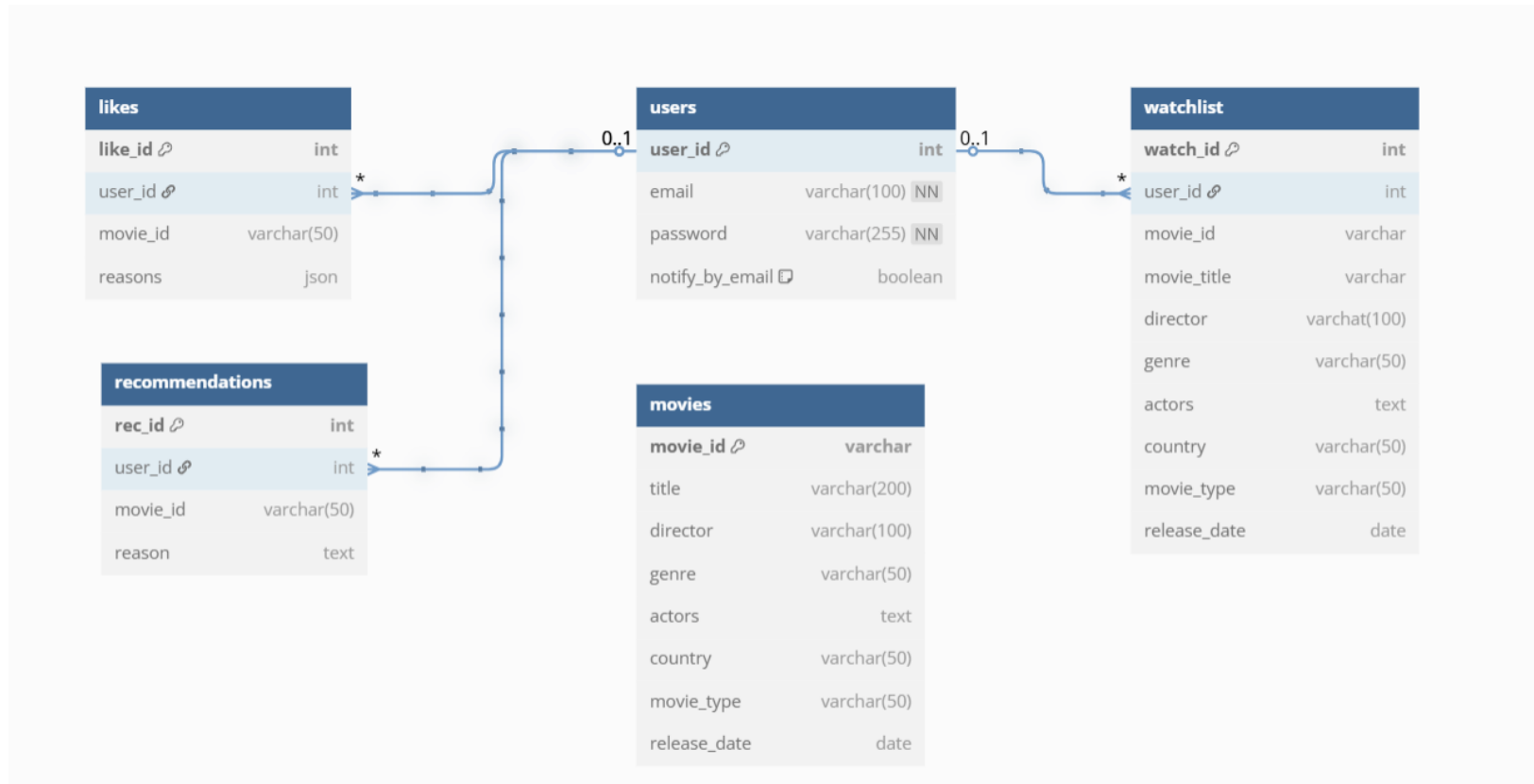
제목: 드래곤 길들이기
개봉일: 2025-06-06
감독: 딘 데블로이스
장르: 액션

제목: 브링 히 백
개봉일: 2025-06-06
감독: 대니 필리프
장르: 공포(호러)

답장 전달

데이터베이스 스키마(UML 스키마)

- 개발 과정에서 계획서의 스키마를 수정했다.



데이터베이스 스키마(Relational 스키마)

- USERS(user_id PK, email, hashed_password, notify_by_email)
- MOVIES(movie_id PK, title, director, genre, actors, country, movie_type, release_date)
- LIKES(like_id PK, user_id FK → USERS.user_id, movie_id, reasons)
- WATCHLIST(watch_id PK, user_id FK → USERS.user_id, movie_id, movie_title, director, genre, actors, country, movie_type, release_date)
- RECOMMENDATIONS(rec_id PK, user_id FK → USERS.user_id, movie_id, reason)

SQL문 – 주요 테이블 생성

```
CREATE TABLE users (  
    user_id SERIAL PRIMARY KEY,  
    email VARCHAR(100) UNIQUE NOT NULL,  
    hashed_password VARCHAR(255) NOT NULL,  
    notify_by_email BOOLEAN DEFAULT TRUE  
);
```

```
CREATE TABLE movies (  
    movie_id VARCHAR(50) PRIMARY KEY,  
    title VARCHAR(200),  
    director VARCHAR(100),  
    genre VARCHAR(50),  
    actors TEXT,  
    country VARCHAR(50),  
    movie_type VARCHAR(50),  
    release_date DATE  
);
```

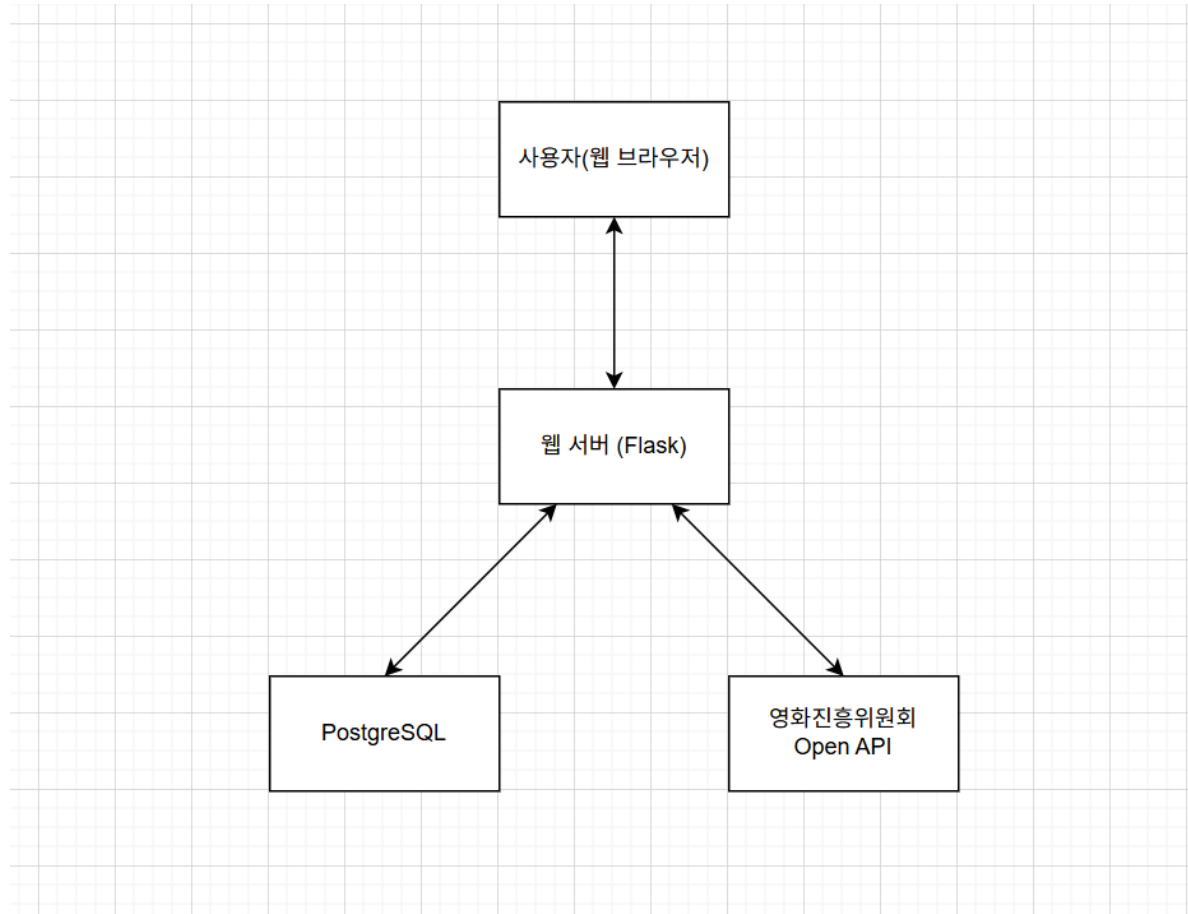
```
CREATE TABLE watchlist (  
    watch_id SERIAL PRIMARY KEY,  
    user_id INTEGER REFERENCES users(user_id),  
    movie_id VARCHAR(50),  
    movie_title VARCHAR(200),  
    director VARCHAR(100),  
    genre VARCHAR(50),  
    actors TEXT,  
    country VARCHAR(50),  
    movie_type VARCHAR(50),  
    release_date DATE,  
    notified BOOLEAN DEFAULT FALSE  
);
```

```
CREATE TABLE likes (  
    like_id SERIAL PRIMARY KEY,  
    user_id INTEGER REFERENCES users(user_id),  
    movie_id VARCHAR(50),  
    reasons JSON  
);  
  
CREATE TABLE recommendations (  
    rec_id SERIAL PRIMARY KEY,  
    user_id INTEGER REFERENCES users(user_id),  
    movie_id VARCHAR(50),  
    reason TEXT  
);
```

SQL문 – 주요 기능을 위한 query

<pre>-- 회원가입 INSERT INTO users (email, hashed_password, notify_by_email) VALUES ('user@example.com', 'hashed_password', true); -- 로그인 시 사용자 조회 SELECT * FROM users WHERE email = 'user@example.com' AND hashed_password = 'hashed_password'; -- 개봉 예정작 조회 (오늘 이후) SELECT * FROM movies WHERE release_date > CURRENT_DATE ORDER BY release_date ASC; -- 사용자의 좋아요 목록 조회 SELECT movie_id, reasons FROM likes WHERE user_id = 1; -- 좋아요 등록 INSERT INTO likes (user_id, movie_id, reasons) VALUES (1, '20240123', '['"감독", "장르"]::json);</pre>	<pre>-- 좋아요 취소 DELETE FROM likes WHERE user_id = 1 AND movie_id = '20240123'; -- 찜 목록 조회 SELECT * FROM watchlist WHERE user_id = 1; -- 찜 추가 INSERT INTO watchlist (user_id, movie_id, movie_title, director, genre, actors, country, movie_type, release_date) VALUES (1, '20240501', '영화제목', '감독명', '장르', '배우들', '한국', '개봉작', '2024-05-25'); -- 찜 삭제 DELETE FROM watchlist WHERE user_id = 1 AND movie_id = '20240501';</pre>	<pre>-- 추천 영화 등록 INSERT INTO recommendations (user_id, movie_id, reason) VALUES (1, '20240501', '감독이 사용자가 좋아한 감독과 동일합니다.');</pre> <pre>-- 추천 영화 목록 조회 (영화 제목 포함) SELECT r.movie_id, m.title, r.reason FROM recommendations r JOIN movies m ON r.movie_id = m.movie_id WHERE r.user_id = 1; -- 추천 영화 삭제 DELETE FROM recommendations WHERE user_id = 1 AND movie_id = '20240501'; -- 회원 탈퇴 시 관련 데이터 삭제 (순서 중요) DELETE FROM likes WHERE user_id = 1; DELETE FROM watchlist WHERE user_id = 1; DELETE FROM recommendations WHERE user_id = 1; DELETE FROM users WHERE user_id = 1;</pre>
--	---	---

시스템 구조



개발 환경

- 언어 :
 - Python (백엔드)
 - HTML / CSS / JavaScript (프론트엔드)
- 프레임워크 / 라이브러리 :
 - Flask (백엔드 웹 프레임워크)
 - SQLAlchemy (ORM)
 - psycopg2 (PostgreSQL 드라이버)
- 데이터베이스 :
 - PostgreSQL
- 개발 및 배포 환경 :
 - Docker & Docker Compose
 - Nginx (Reverse Proxy 설정)
 - Oracle Cloud Infrastructure – Ubuntu 인스턴스
- IDE :
 - cursor, vscode
- LLM :
 - ChatGPT
- 외부 API :
 - 영화진흥위원회 OpenAPI